

PART A

(PART A : TO BE REFERRED BY STUDENTS)

Experiment No. 2 (A)

Aim : To design and implement second pass of a two pass assembler for IBM 360/370 Processor

Objective: Develop a program to implement second pass:

- a. To generate Base table
- b. To generate machine code

Outcome: Students are able to design and implement pass 2 of two pass assembler. **Theory:**

Pass 2: Purpose - To generate object program

- 1) Look up value of symbols (STGET)
- 2) Generate instruction (MOTGET2)
- 3) Generate data (for DC, DS)
- 4) Process pseudo ops (POT, GET2)

Data Structures:

- 1) Copy of source program from Pass1
- 2) Location counter
- 3) MOT which gives the length, mnemonic format opcode
- 4) POT which gives mnemonic & action to be taken
- 5) Symbol table from Pass1
- 6) Base table which indicates the register to be used or base register
- 7) A work space INST to hold the instruction & its parts
- 8) A work space PRINT LINE, to produce printed listing
- 9) A work space PUNCH CARD for converting instruction into format needed by loader
- 10) An output deck of assembled instructions needed by loader.

Format of database:

Base Table:

Assembler uses this table to generate proper base register reference in machine instructions and to compute offset. Then the offset is calculated as:

$$\text{offset} = \text{value of symbol from ST} - \text{contents of base register}$$

4 bytes per entry

	Availability indicator (1-byte) characters	Designated relative address contents of base register (3 bytes= 24 bits address) hexadecimal
1	"N"	
	"N"	
2...		
	"Y"	

Y : Register specified in USING pseudo-op

N: Register never specified in USING pseudo-op or made unavailable by DROP pseudo-op.

Let us consider the same example as experiment no. 1 and the base table after statement 2:

Base register	Contents
15	0

After statement 7:

Base register	Contents
15	10

Code after pass2:

stmt no	Relative address	Statement		
3	0		A	1, 12 (0,15)
4	4		A	2, 16(0,15)
6	8		A	3, 10(0,15)
8	12		4	
9	16		5	
10		-		

Algorithm:

1. Initialize the location counter as: LC=0
2. Read the statement from source program
3. Examine the op-code field: If match found in MOT then
 - a. From the MOT entry determine the length field i.e. L=length, binary op-code and format of the instruction.

Different instruction format requires different processing as described below:

1. RR Instruction : (Register to Register)

Both of the register specification fields are evaluated and placed into second byte of RR instruction

2. RX Instruction : (Register to Index)

Both of the register and index fields are evaluated and processed similar to RR instruction. The storage address operand is evaluated to generate effective address (EA). The BT is examined to find the base register. Then the displacement is

determined as:

D=EA- Contents of base register.

The other instruction formats are processed in similar manner to RR and RX.

- b. Finally the base register and displacement specification are assembled in third and fourth bytes of instruction.
- c. The current value of location counter is incremented by length of instruction.

4. If match found in POT then

- a. If it is EQU pseudo-op then EQU card is printed in the listings.
- b. If it is USING pseudo-op then the corresponding BT entry is marked as available.
- c. If it is DROP pseudo-op then the corresponding BT entry is marked as unavailable.
- d. If it is DS or DC pseudo-op then various conversions are done depending on the data type and symbols are evaluated. Location counter is updated by length of data.
- e. END pseudo-op indicates end of source program and then pass2 is terminated.

Before that if any literals are remaining then the code is generated for them. 5. After assembling the instruction it is put in the format required by loader. 6. Finally a listing is printed which consist of copy of source card, its storage location and hexadecimal representation.

7. Go to step 2.

PART B

(PART B : TO BE COMPLETED BY STUDENTS)

(Students must submit the soft copy as per following segments within two hours of the practical. The soft copy must be uploaded at the end of the practical)

Roll. No. C-54	Name: Quazi Md Moinuddin
Class: TE Comps C	Batch: C-3

Date of Experiment: 29.01.2025	Date of Submission: 28.02.2025
Grade:	

B.1 Software Code written by student:

C Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(void)
{
    char a[10], ad[10], label[10], opcode[10], operand[10], symbol[10];
    int st, diff, i, address = 0, add, len, actual_len, finaddr, prevaddr, j = 0;
    char mnemonic[15][15] = {"LDA", "STA", "LDCH", "STCH"};
    char code[15][15] = {"33", "44", "53", "57"};
    FILE *fp1, *fp2, *fp3, *fp4;
    int ch; // Use int for fgetc() value

    /* Open files with error checking */
    fp1 = fopen("ASSMLIST.DAT", "w");
    if (fp1 == NULL) { perror("Error opening ASSMLIST.DAT"); exit(1); }

    fp2 = fopen("SYMTAB.DAT", "r");
    if (fp2 == NULL) { perror("Error opening SYMTAB.DAT"); exit(1); }

    fp3 = fopen("INTERMED.DAT", "r");
    if (fp3 == NULL) { perror("Error opening INTERMED.DAT"); exit(1); }

    fp4 = fopen("OBJCODE.DAT", "w");
    if (fp4 == NULL) { perror("Error opening OBJCODE.DAT"); exit(1); }

    /* Read the first line from the intermediate file */
    fscanf(fp3, "%s %s %s", label, opcode, operand);

    /* Process the intermediate file to determine the final address */
    while (strcmp(opcode, "END") != 0)
    {
        prevaddr = address;
        fscanf(fp3, "%d %s %s %s", &address, label, opcode, operand);
    }
    finaddr = address;
    fclose(fp3);
}
```

```

/* Reopen the intermediate file for a second pass */
fp3 = fopen("INTERMED.DAT", "r");
if (fp3 == NULL) { perror("Error reopening INTERMED.DAT"); exit(1); }

fscanf(fp3, "%s %s %s", label, opcode, operand);
if (strcmp(opcode, "START") == 0)
{
    fprintf(fp1, "%t%s\t%s\t%s\n", label, opcode, operand);
    fprintf(fp4, "H^%s^00%s^00%d\n", label, operand, finaddr);

    fscanf(fp3, "%d %s %s %s", &address, label, opcode, operand);
    st = address;
    diff = prevaddr - st;
    fprintf(fp4, "T^00%d^%d", address, diff);
}

while (strcmp(opcode, "END") != 0)
{
    if (strcmp(opcode, "BYTE") == 0)
    {
        fprintf(fp1, "%d\t%s\t%s\t%s\t", address, label, opcode, operand);
        len = strlen(operand);
        actual_len = len - 3; // Adjust for the format (e.g., C'EOF')
        fprintf(fp4, "^");
        for (i = 2; i < (actual_len + 2); i++)
        {
            /* Convert the character to hexadecimal using sprintf */
            sprintf(ad, "%X", operand[i]);
            fprintf(fp1, "%s", ad);
            fprintf(fp4, "%s", ad);
        }
        fprintf(fp1, "\n");
    }
    else if (strcmp(opcode, "WORD") == 0)
    {
        /* Convert operand string to integer and back to string */
        sprintf(a, "%d", atoi(operand));
        fprintf(fp1, "%d\t%s\t%s\t%s\t000000%s\n", address, label, opcode,
operand, a);
        fprintf(fp4, "^000000%s", a);
    }
    else if ((strcmp(opcode, "RESB") == 0) || (strcmp(opcode, "RESW") == 0))
    {
        fprintf(fp1, "%d\t%s\t%s\t%s\n", address, label, opcode, operand);
    }
    else
    {

```

```

/* Reset j for each new instruction */
j = 0;
while (strcmp(opcode, mnemonic[j]) != 0 && j < 15)
    j++;
if (strcmp(operand, "COPY") == 0)
{
    fprintf(fp1, "%d\t%s\t%s\t%s\t%s0000\n", address, label, opcode,
operand, code[j]);
}
else
{
    rewind(fp2);
    fscanf(fp2, "%s %d", symbol, &add);
    while (strcmp(operand, symbol) != 0)
        fscanf(fp2, "%s %d", symbol, &add);
    fprintf(fp1, "%d\t%s\t%s\t%s\t%d\n", address, label, opcode,
operand, code[j], add);
    fprintf(fp4, "^%s%d", code[j], add);
}
}
fscanf(fp3, "%d %s %s %s", &address, label, opcode, operand);
}

```

```

fprintf(fp1, "%d\t%s\t%s\t%s\n", address, label, opcode, operand);
fprintf(fp4, "\nE^00%d", st);
printf("\n Intermediate file is converted into object code");

```

```

/* Close all opened files explicitly */

```

```

fclose(fp1);
fclose(fp2);
fclose(fp3);
fclose(fp4);

```

```

/* Display the contents of the files */

```

```

printf("\n\nThe contents of Intermediate file:\n\n\t");
fp3 = fopen("INTERMED.DAT", "r");
if (fp3 == NULL) { perror("Error opening INTERMED.DAT"); exit(1); }
while ((ch = fgetc(fp3)) != EOF)
{
    printf("%c", ch);
}
fclose(fp3);

```

```

printf("\n\nThe contents of Symbol Table :\n\n");
fp2 = fopen("SYMTAB.DAT", "r");
if (fp2 == NULL) { perror("Error opening SYMTAB.DAT"); exit(1); }
while ((ch = fgetc(fp2)) != EOF)

```

```

    {
        printf("%c", ch);
    }
    fclose(fp2);

    printf("\n\nThe contents of Output file :\n\n");
    fp1 = fopen("ASSMLIST.DAT", "r");
    if (fp1 == NULL) { perror("Error opening ASSMLIST.DAT"); exit(1); }
    while ((ch = fgetc(fp1)) != EOF)
    {
        printf("%c", ch);
    }
    fclose(fp1);

    printf("\n\nThe contents of Object code file :\n\n");
    fp4 = fopen("OBJCODE.DAT", "r");
    if (fp4 == NULL) { perror("Error opening OBJCODE.DAT"); exit(1); }
    while ((ch = fgetc(fp4)) != EOF)
    {
        printf("%c", ch);
    }
    fclose(fp4);

    return 0;
}

```

INPUT :

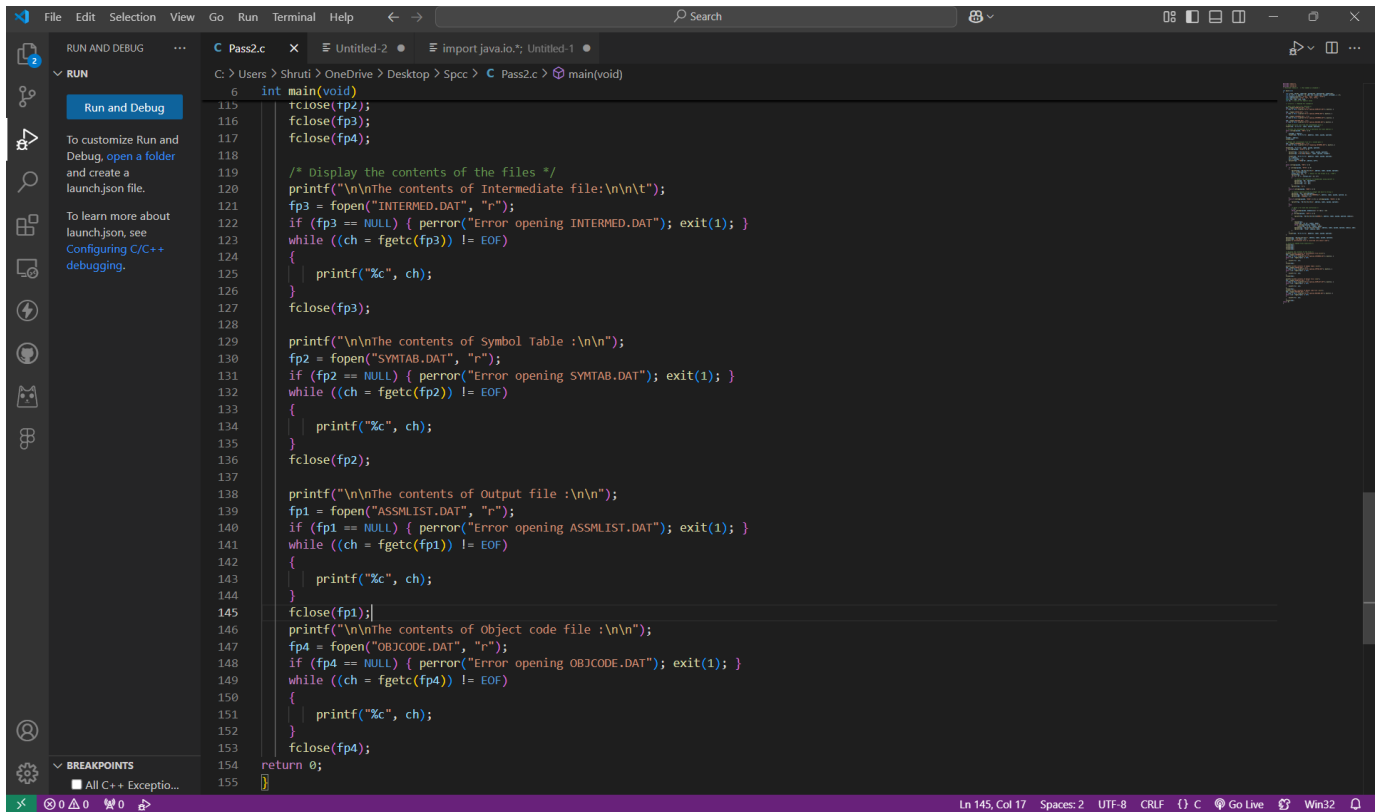
```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  // #include <conio.h> // Not needed in standard C
5
6  int main(void)
7  {
8      char a[10], ad[10], label[10], opcode[10], operand[10], symbol[10];
9      int st, diff, i, address = 0, add, len, actual_len, finaddr, prevaddr, j = 0;
10     char mnemonic[15][15] = {"LDA", "STA", "LDCH", "STCH"};
11     char code[15][15] = {"33", "44", "53", "57"};
12     FILE *fp1, *fp2, *fp3, *fp4;
13     int ch; // Use int for fgetc() value
14
15     // clrscr(); // Removed for standard C
16
17     /* Open files with error checking */
18     fp1 = fopen("ASSMLIST.DAT", "w");
19     if (fp1 == NULL) { perror("Error opening ASSMLIST.DAT"); exit(1); }
20
21     fp2 = fopen("SYMTAB.DAT", "r");
22     if (fp2 == NULL) { perror("Error opening SYMTAB.DAT"); exit(1); }
23
24     fp3 = fopen("INTERMED.DAT", "r");
25     if (fp3 == NULL) { perror("Error opening INTERMED.DAT"); exit(1); }
26
27     fp4 = fopen("OBJCODE.DAT", "w");
28     if (fp4 == NULL) { perror("Error opening OBJCODE.DAT"); exit(1); }
29
30     /* Read the first line from the intermediate file */
31     fscanf(fp3, "%s %s %s", label, opcode, operand);
32
33     /* Process the intermediate file to determine the final address */
34     while (strcmp(opcode, "END") != 0)
35     {
36         prevaddr = address;
37         fscanf(fp3, "%d %s %s", &address, label, opcode, operand);
38     }
39     finaddr = address;
40     fclose(fp3);
41
42     /* Reopen the intermediate file for a second pass */

```

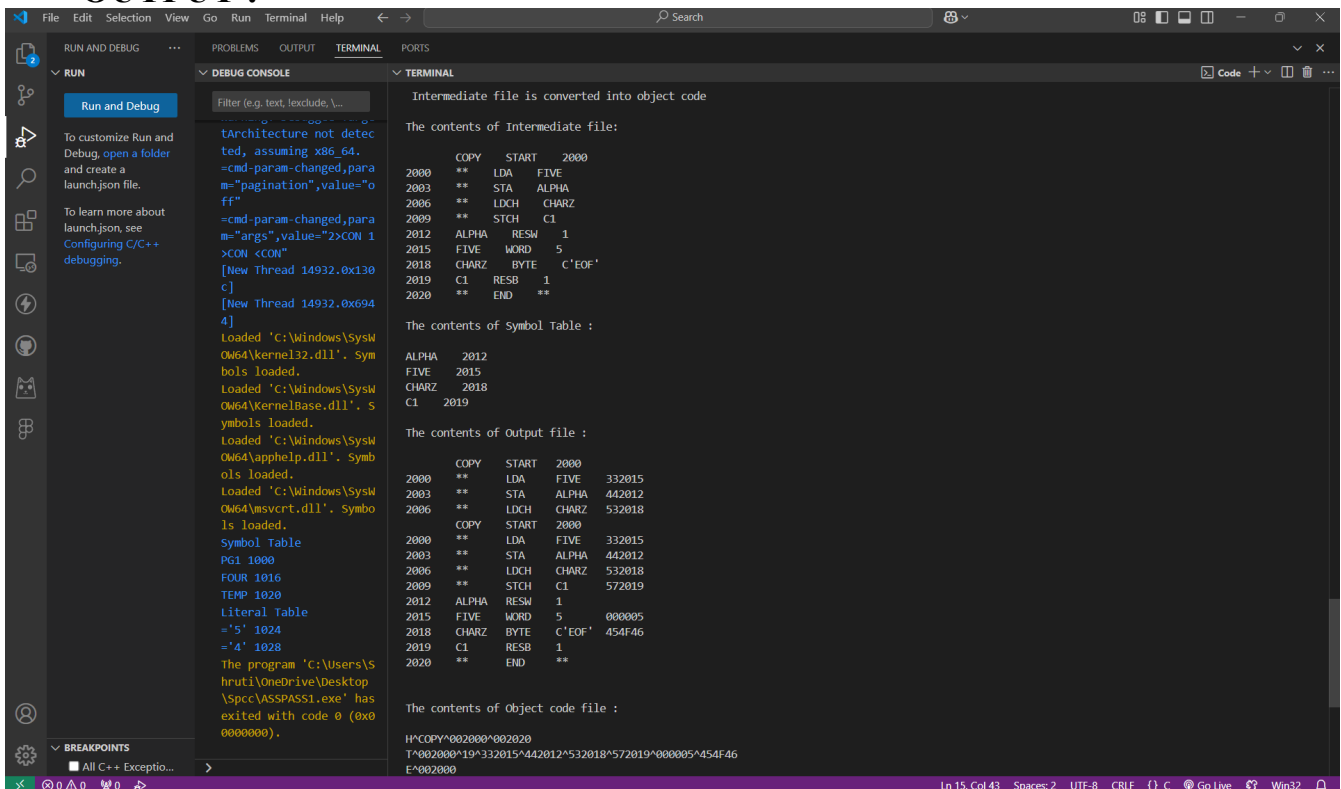
```
File Edit Selection View Go Run Terminal Help Search
C: > Users > Shruti > OneDrive > Desktop > Spcc > C Pass2.c > main(void)
6 int main(void)
43 /* Convert the intermediate file into a second pass */
44 fp3 = fopen("INTERMED.DAT", "r");
45 if (fp3 == NULL) { perror("Error reopening INTERMED.DAT"); exit(1); }
46
47 fscanf(fp3, "%s %s %s", label, opcode, operand);
48 if (strcmp(opcode, "START") == 0)
49 {
50     fprintf(fp1, "\t%s\t%s\t%s\n", label, opcode, operand);
51     fprintf(fp4, "H%s^0000^0000\n", label, operand, finaddr);
52
53     fscanf(fp3, "%d %s %s %s", &address, label, opcode, operand);
54     st = address;
55     diff = prevaddr - st;
56     fprintf(fp4, "T^0000^0000", address, diff);
57 }
58
59 while (strcmp(opcode, "END") != 0)
60 {
61     if (strcmp(opcode, "BYTE") == 0)
62     {
63         fprintf(fp1, "%d\t%s\t%s\t%s\n", address, label, opcode, operand);
64         len = strlen(operand);
65         actual_len = len - 3; // Adjust for the format (e.g., C'EOF')
66         fprintf(fp4, "^");
67         for (i = 2; i < (actual_len + 2); i++)
68         {
69             /* Convert the character to hexadecimal using sprintf */
70             sprintf(ad, "%X", operand[i]);
71             fprintf(fp1, "%s", ad);
72             fprintf(fp4, "%s", ad);
73         }
74         fprintf(fp1, "\n");
75     }
76     else if (strcmp(opcode, "WORD") == 0)
77     {
78         /* Convert operand string to integer and back to string */
79         sprintf(a, "%d", atoi(operand));
80         fprintf(fp1, "%d\t%s\t%s\t%s\t00000000\n", address, label, opcode, operand, a);
81         fprintf(fp4, "^00000000", a);
82     }
83     else if ((strcmp(opcode, "RESEB") == 0) || (strcmp(opcode, "RESW") == 0))
84     {
85         fprintf(fp1, "%d\t%s\t%s\t%s\n", address, label, opcode, operand);
86     }
87     else
88     {
89         /* Reset j for each new instruction */
90         j = 0;
91         while (strcmp(opcode, mnemonic[j]) != 0 && j < 15)
92             j++;
93         if (strcmp(operand, "COPY") == 0)
94         {
95             fprintf(fp1, "%d\t%s\t%s\t%s\t%s0000\n", address, label, opcode, operand, code[j]);
96         }
97         else
98         {
99             rewind(fp2);
100             fscanf(fp2, "%s %d", symbol, &add);
101             while (strcmp(operand, symbol) != 0)
102                 fscanf(fp2, "%s %d", symbol, &add);
103             fprintf(fp1, "%d\t%s\t%s\t%s\t%s\n", address, label, opcode, operand, code[j], add);
104             fprintf(fp4, "%s%d", code[j], add);
105         }
106     }
107     fscanf(fp3, "%d %s %s %s", &address, label, opcode, operand);
108 }
109
110 fprintf(fp1, "%d\t%s\t%s\t%s\n", address, label, opcode, operand);
111 fprintf(fp4, "\nE^0000", st);
112 printf("\nIntermediate file is converted into object code");
113
114 /* Close all opened files explicitly */
115 fclose(fp1);
116 fclose(fp2);
117 fclose(fp3);
118 fclose(fp4);
119
120 /* Display the contents of the files */
121 printf("\n\nThe contents of Intermediate file:\n\n\t");
122 fp3 = fopen("INTERMED.DAT", "r");
```

```
File Edit Selection View Go Run Terminal Help Search
C: > Users > Shruti > OneDrive > Desktop > Spcc > C Pass2.c > main(void)
58 int main(void)
59 while (strcmp(opcode, "END") != 0)
60 {
61     if (strcmp(opcode, "BYTE") == 0)
62     {
63         fprintf(fp1, "%d\t%s\t%s\t%s\n", address, label, opcode, operand);
64         len = strlen(operand);
65         actual_len = len - 3; // Adjust for the format (e.g., C'EOF')
66         fprintf(fp4, "^");
67         for (i = 2; i < (actual_len + 2); i++)
68         {
69             /* Convert the character to hexadecimal using sprintf */
70             sprintf(ad, "%X", operand[i]);
71             fprintf(fp1, "%s", ad);
72             fprintf(fp4, "%s", ad);
73         }
74         fprintf(fp1, "\n");
75     }
76     else if (strcmp(opcode, "WORD") == 0)
77     {
78         /* Convert operand string to integer and back to string */
79         sprintf(a, "%d", atoi(operand));
80         fprintf(fp1, "%d\t%s\t%s\t%s\t00000000\n", address, label, opcode, operand, a);
81         fprintf(fp4, "^00000000", a);
82     }
83     else if ((strcmp(opcode, "RESEB") == 0) || (strcmp(opcode, "RESW") == 0))
84     {
85         fprintf(fp1, "%d\t%s\t%s\t%s\n", address, label, opcode, operand);
86     }
87     else
88     {
89         /* Reset j for each new instruction */
90         j = 0;
91         while (strcmp(opcode, mnemonic[j]) != 0 && j < 15)
92             j++;
93         if (strcmp(operand, "COPY") == 0)
94         {
95             fprintf(fp1, "%d\t%s\t%s\t%s\t%s0000\n", address, label, opcode, operand, code[j]);
96         }
97         else
98         {
99             rewind(fp2);
100             fscanf(fp2, "%s %d", symbol, &add);
101             while (strcmp(operand, symbol) != 0)
102                 fscanf(fp2, "%s %d", symbol, &add);
103             fprintf(fp1, "%d\t%s\t%s\t%s\t%s\n", address, label, opcode, operand, code[j], add);
104             fprintf(fp4, "%s%d", code[j], add);
105         }
106     }
107     fscanf(fp3, "%d %s %s %s", &address, label, opcode, operand);
108 }
109
110 fprintf(fp1, "%d\t%s\t%s\t%s\n", address, label, opcode, operand);
111 fprintf(fp4, "\nE^0000", st);
112 printf("\nIntermediate file is converted into object code");
113
114 /* Close all opened files explicitly */
115 fclose(fp1);
116 fclose(fp2);
117 fclose(fp3);
118 fclose(fp4);
119
120 /* Display the contents of the files */
121 printf("\n\nThe contents of Intermediate file:\n\n\t");
122 fp3 = fopen("INTERMED.DAT", "r");
```

```
6 int main(void)
115 {
116     fclose(fp2);
117     fclose(fp3);
118     fclose(fp4);
119
120     /* Display the contents of the files */
121     printf("\n\nThe contents of Intermediate file:\n\n\t");
122     fp3 = fopen("INTERMED.DAT", "r");
123     if (fp3 == NULL) { perror("Error opening INTERMED.DAT"); exit(1); }
124     while ((ch = fgetc(fp3)) != EOF)
125     {
126         printf("%c", ch);
127     }
128     fclose(fp3);
129
130     printf("\n\nThe contents of Symbol Table :\n\n");
131     fp2 = fopen("SYMTAB.DAT", "r");
132     if (fp2 == NULL) { perror("Error opening SYMTAB.DAT"); exit(1); }
133     while ((ch = fgetc(fp2)) != EOF)
134     {
135         printf("%c", ch);
136     }
137     fclose(fp2);
138
139     printf("\n\nThe contents of Output file :\n\n");
140     fp1 = fopen("ASSMLIST.DAT", "r");
141     if (fp1 == NULL) { perror("Error opening ASSMLIST.DAT"); exit(1); }
142     while ((ch = fgetc(fp1)) != EOF)
143     {
144         printf("%c", ch);
145     }
146     fclose(fp1);
147     printf("\n\nThe contents of Object code file :\n\n");
148     fp4 = fopen("OBJCODE.DAT", "r");
149     if (fp4 == NULL) { perror("Error opening OBJCODE.DAT"); exit(1); }
150     while ((ch = fgetc(fp4)) != EOF)
151     {
152         printf("%c", ch);
153     }
154     fclose(fp4);
155     return 0;
}
```

OUTPUT :



```
Architecture not detected, assuming x86_64.
=cmd-param-changed,para
m="pagination",value="o
ff"
=cmd-param-changed,para
m="args",value="2>CON 1
>CON <CON"
[New Thread 14932.0x130
c]
[New Thread 14932.0x694
4]
Loaded 'C:\Windows\SysW
Ow64\kernel32.dll'. Sym
bols loaded.
Loaded 'C:\Windows\SysW
Ow64\kernelBase.dll'. S
ymbols loaded.
Loaded 'C:\Windows\SysW
Ow64\apphelp.dll'. Symb
ols loaded.
Loaded 'C:\Windows\SysW
Ow64\msvcrt.dll'. Symbo
ls loaded.
Symbol Table
Pg1 1000
FOUR 1016
TEMP 1020
Literal Table
='5' 1024
='4' 1028
The program 'C:\Users\S
hruti\OneDrive\Desktop
\Spcc\ASSPASS1.exe' has
exited with code 0 (0x0
0000000).
```

COPY	START	2000
** LDA	FIVE	332015
** STA	ALPHA	442012
** LDCH	CHARZ	532018
** STCH	C1	572019
ALPHA	RESW	1
FIVE	WORD	5
CHARZ	BYTE	C'EOF'
C1	RESB	1
** END	**	

ALPHA	2012
FIVE	2015
CHARZ	2018
C1	2019

COPY	START	2000
** LDA	FIVE	332015
** STA	ALPHA	442012
** LDCH	CHARZ	532018
** STCH	C1	572019
ALPHA	RESW	1
FIVE	WORD	5
CHARZ	BYTE	C'EOF'
C1	RESB	1
** END	**	

ALPHA	2012
FIVE	2015
CHARZ	2018
C1	2019

COPY	START	2000
** LDA	FIVE	332015
** STA	ALPHA	442012
** LDCH	CHARZ	532018
** STCH	C1	572019
ALPHA	RESW	1
FIVE	WORD	5
CHARZ	BYTE	C'EOF'
C1	RESB	1
** END	**	

ALPHA	2012
FIVE	2015
CHARZ	2018
C1	2019

COPY	START	2000
** LDA	FIVE	332015
** STA	ALPHA	442012
** LDCH	CHARZ	532018
** STCH	C1	572019
ALPHA	RESW	1
FIVE	WORD	5
CHARZ	BYTE	C'EOF'
C1	RESB	1
** END	**	

ALPHA	2012
FIVE	2015
CHARZ	2018
C1	2019

COPY	START	2000
** LDA	FIVE	332015
** STA	ALPHA	442012
** LDCH	CHARZ	532018
** STCH	C1	572019
ALPHA	RESW	1
FIVE	WORD	5
CHARZ	BYTE	C'EOF'
C1	RESB	1
** END	**	

ALPHA	2012
FIVE	2015
CHARZ	2018
C1	2019

COPY	START	2000
** LDA	FIVE	332015
** STA	ALPHA	442012
** LDCH	CHARZ	532018
** STCH	C1	572019
ALPHA	RESW	1
FIVE	WORD	5
CHARZ	BYTE	C'EOF'
C1	RESB	1
** END	**	

ALPHA	2012
FIVE	2015
CHARZ	2018
C1	2019

COPY	START	2000
** LDA	FIVE	332015
** STA	ALPHA	442012
** LDCH	CHARZ	532018
** STCH	C1	572019
ALPHA	RESW	1
FIVE	WORD	5
CHARZ	BYTE	C'EOF'
C1	RESB	1
** END	**	

ALPHA	2012
FIVE	2015
CHARZ	2018
C1	2019

COPY	START	2000
** LDA	FIVE	332015
** STA	ALPHA	442012
** LDCH	CHARZ	532018
** STCH	C1	572019
ALPHA	RESW	1
FIVE	WORD	5
CHARZ	BYTE	C'EOF'
C1	RESB	1
** END	**	

ALPHA	2012
FIVE	2015
CHARZ	2018
C1	2019

COPY	START	2000
** LDA	FIVE	332015
** STA	ALPHA	442012
** LDCH	CHARZ	532018
** STCH	C1	572019
ALPHA	RESW	1
FIVE	WORD	5
CHARZ	BYTE	C'EOF'
C1	RESB	1
** END	**	

ALPHA	2012
FIVE	2015
CHARZ	2018
C1	2019

COPY	START	2000
** LDA	FIVE	332015
** STA	ALPHA	442012
** LDCH	CHARZ	532018
** STCH	C1	572019
ALPHA	RESW	1
FIVE	WORD	5
CHARZ	BYTE	C'EOF'
C1	RESB	1
** END	**	

ALPHA	2012
FIVE	2015
CHARZ	2018
C1	2019

COPY	START	2000
** LDA	FIVE	332015
** STA	ALPHA	442012
** LDCH	CHARZ	532018
** STCH	C1	572019
ALPHA	RESW	1
FIVE	WORD	5
CHARZ	BYTE	C'EOF'
C1	RESB	1
** END	**	

ALPHA	2012
FIVE	2015
CHARZ	2018
C1	2019

COPY	START	2000
** LDA	FIVE	332015
** STA	ALPHA	442012
** LDCH	CHARZ	532018
** STCH	C1	572019
ALPHA	RESW	1
FIVE	WORD	5
CHARZ	BYTE	C'EOF'
C1	RESB	1
** END	**	

ALPHA	2012
FIVE	2015
CHARZ	2018
C1	2019

COPY	START	2000
** LDA	FIVE	332015
** STA	ALPHA	442012
** LDCH	CHARZ	532018
** STCH	C1	572019
ALPHA	RESW	1
FIVE	WORD	5
CHARZ	BYTE	C'EOF'
C1	RESB	1
** END	**	

ALPHA	2012
FIVE	2015
CHARZ	2018
C1	2019

COPY	START	2000
** LDA	FIVE	332015
** STA	ALPHA	442012
** LDCH	CHARZ	532018
** STCH	C1	572019
ALPHA	RESW	1
FIVE	WORD	5
CHARZ	BYTE	C'EOF'
C1	RESB	1
** END	**	

ALPHA	2012
FIVE	2015
CHARZ	2018
C1	2019

COPY	START	2000
** LDA	FIVE	332015
** STA	ALPHA	442012
** LDCH	CHARZ	532018
** STCH	C1	572019
ALPHA	RESW	1
FIVE	WORD	5
CHARZ	BYTE	C'EOF'
C1	RESB	1
** END	**	

ALPHA	2012
FIVE	2015
CHARZ	2018
C1	2019

COPY	START	2000
** LDA	FIVE	332015
** STA	ALPHA	442012
** LDCH	CHARZ	532018
** STCH	C1	572019
ALPHA	RESW	1
FIVE	WORD	5
CHARZ	BYTE	C'EOF'
C1	RESB	1
** END	**	

ALPHA	2012
FIVE	2015
CHARZ	2018
C1	2019

COPY	START	2000
** LDA	FIVE	332015
** STA	ALPHA	442012
** LDCH	CHARZ	532018
** STCH	C1	572019
ALPHA	RESW	1
FIVE	WORD	5
CHARZ	BYTE	C'EOF'
C1	RESB	1
** END	**	

ALPHA	2012
FIVE	2015
CHARZ	2018
C1	2019

COPY	START	2000
** LDA	FIVE	332015
** STA	ALPHA	442012
** LDCH	CHARZ	532018
** STCH	C1	572019
ALPHA	RESW	1
FIVE	WORD	5
CHARZ	BYTE	C'EOF'
C1	RESB	1
** END	**	

ALPHA	2012
FIVE	2015
CHARZ	2018
C1	2019

COPY	START	2000
** LDA	FIVE	332015
** STA	ALPHA	442012
** LDCH	CHARZ	532018
** STCH	C1	572019
ALPHA	RESW	1
FIVE	WORD	5
CHARZ	BYTE	C'EOF'
C1	RESB	1
** END	**	

ALPHA	2012
FIVE	2015
CHARZ	2018
C1	2019

COPY	START	2000
** LDA	FIVE	332015
** STA	ALPHA	442012
** LDCH	CHARZ	532018
** STCH	C1	572019
ALPHA	RESW	1
FIVE	WORD	5
CHARZ	BYTE	C'EOF'
C1	RESB	1
** END	**	

ALPHA	2012
FIVE	2015
CHARZ	2018
C1	2019

COPY	START	2000
** LDA	FIVE	332015
** STA	ALPHA	442012
** LDCH	CHARZ	532018
** STCH	C1	572019
ALPHA	RESW	1
FIVE	WORD	5
CHARZ	BYTE	C'EOF'
C1	RESB	1
** END	**	

ALPHA	2012
FIVE	2015
CHARZ	2018
C1	2019

COPY	START	2000
** LDA	FIVE	332015
** STA	ALPHA	442012
** LDCH	CHARZ	532018
** STCH	C1	572019
ALPHA	RESW	1
FIVE	WORD	5
CHARZ	BYTE	C'EOF'
C1	RESB	1
** END	**	

ALPHA	2012
FIVE	2015
CHARZ	2018
C1	2019

COPY	START	2000
** LDA	FIVE	332015
** STA	ALPHA	442012
** LDCH	CHARZ	532018
** STCH	C1	572019
ALPHA	RESW	1
FIVE	WORD	5
CHARZ	BYTE	C'EOF'
C1	RESB	1
** END	**	

ALPHA	2012
FIVE	2015
CHARZ	2018
C1	2019

COPY	START	2000
** LDA	FIVE	332015
** STA	ALPHA	442012
** LDCH	CHARZ	532018
** STCH	C1	572019
ALPHA	RESW	1
FIVE	WORD	5
CHARZ	BYTE	C'EOF'
C1	RESB	1
** END	**	

ALPHA	2012
FIVE	2015
CHARZ	2018
C1	2019

COPY	START	2000
** LDA	FIVE	332015
** STA	ALPHA	442012
** LDCH	CHARZ	532018
** STCH	C1	572019
ALPHA	RESW	1
FIVE	WORD	5
CHARZ	BYTE	C'EOF'
C1	RESB	1
** END	**	

ALPHA	2012
FIVE	2015
CHARZ	2018
C1	2019

COPY	START	2000
** LDA	FIVE	332015
** STA	ALPHA	442012
** LDCH	CHARZ	532018
** STCH	C1	572019
ALPHA	RESW	1
FIVE	WORD	5
CHARZ	BYTE	C'EOF'
C1	RESB	1
** END	**	

ALPHA	2012
FIVE	2015
CHARZ	2018
C1	2019

COPY	START	2000
** LDA	FIVE	332015
** STA	ALPHA	442012
** LDCH	CHARZ	532018
** STCH	C1	572019
ALPHA	RESW	1
FIVE	WORD	5
CHARZ	BYTE	C'EOF'
C1	RESB	1
** END	**	

ALPHA	2012
FIVE	2015
CHARZ	2018
C1	2019

COPY	START	2000
** LDA	FIVE	332015
** STA	ALPHA	442012
** LDCH	CHARZ	532018
** STCH	C1	572019
ALPHA	RESW	1
FIVE	WORD	5
CHARZ	BYTE	C'EOF'
C1	RESB	1
** END	**	

ALPHA	2012
FIVE	2015
CHARZ	2018
C1	2019

COPY	START	2000
** LDA	FIVE	332015
** STA	ALPHA	442012
** LDCH	CHARZ	532018
** STCH	C1	572019
ALPHA	RESW	1
FIVE	WORD	5
CHARZ	BYTE	C'EOF'
C1	RESB	1
** END	**	

ALPHA	2012
FIVE	2015
CHARZ	2018
C1	2019

COPY	START	2000
** LDA	FIVE	332015
** STA	ALPHA	442012
** LDCH	CHARZ	532018
** STCH	C1	572019
ALPHA	RESW	1
FIVE	WORD	5
CHARZ	BYTE	C'EOF'
C1	RESB	1
** END	**	

ALPHA	2012
FIVE	2015
CHARZ	2018
C1	2019

COPY	START	2000
** LDA	FIVE	332015
** STA	ALPHA	442012
** LDCH	CHARZ	532018
** STCH	C1	572019
ALPHA	RESW	1
FIVE	WORD	5
CHARZ	BYTE	C'EOF'
C1	RESB	1
** END	**	

ALPHA	2012
FIVE	2015
CHARZ	2018
C1	2019

COPY	START	2000
** LDA	FIVE	332015
** STA	ALPHA	442012
** LDCH	CHARZ	532018
** STCH	C1	572019
ALPHA	RESW	1
FIVE	WORD	5
CHARZ	BYTE	C'EOF'
C1	RESB	1

B.3 Observations and learning:

The second pass of a two-pass assembler involves using the Symbol Table and the Forward Reference Table to generate the final machine code or object code for the program. By resolving references to undefined symbols and replacing symbolic operands with their corresponding memory addresses, the assembler can generate correct and functional code. The listing file helps programmers to debug and verify the correctness of their program.

B.4 Conclusion:

Hence, we successfully designed and implemented second pass of a two-pass assembler for IBM 360/370 Processor.

B.5 Question of Curiosity

(To be answered by student based on the practical performed and learning/observations)

A. Give Example of Working of Two Pass Assembler

Ans : Let us consider a simple example of a two-pass assembler for the IBM 360/370 Processor. The program source code is as follows:

```
START 1000
USING *,15
LOOP L 1,=F'5'
A 1,TWO
    ST 1,TEMP
    C 1,=F'4'
    BNZ LOOP
END LOOP
TWO DC 2
TEMP DS 1F
END START
```

In the first pass, the assembler scans the source code and builds a Symbol Table and a Forward Reference Table. The assembler also generates a listing file that includes the original source code, the addresses assigned to each statement, and any error messages or warnings.

The Symbol Table and Forward Reference Table after the first pass are as follows: Symbol Table

```
LOOP 1000
TWO 100C
TEMP 100E
```

Forward Reference Table

```
1 LOOP+1
```

In the second pass, the assembler uses the Symbol Table and the Forward Reference

Table to generate the final machine code or object code for the program. The assembler also generates a new listing file that includes the original source code, the generated code, and any error messages or warnings.

The generated machine code or object code after the second pass is as follows:

Address Machine Code Source Code

1000 5800 015 LOOP L 1,=F'5'

1003 8100 002 A 1,TWO

1006 1100 01C ST 1,TEMP

1009 4100 004 C 1,=F'4'

100C D200 001 TWO DC 2

100E 0000 000 DS 1F

100F 0000 000 END START

The generated machine code or object code is represented in hexadecimal format. The assembler has replaced symbolic operands with their corresponding memory addresses and updated the memory addresses of forward references with the actual memory addresses of the corresponding symbols. The assembler has also added the required assembler directives, such as the START and END directives.

Overall, this example demonstrates how a two-pass assembler works by building a Symbol Table and a Forward Reference Table in the first pass and using them to generate the final machine code or object code in the second pass.

B. Mention the Advantage of assemblers with Multiple Passes? Ans :

- One of the main advantages of Two-Pass Assembler is that many times the first pass of an extreme Two-pass assembler generates the output file which is then read by the second pass.
- The advantage of this is that first pass can record each line of input, along with that the next position of some or all lexemes of that line and some of the results of parsing.
- For example, in an assembly-level language, each line can be sent to the other line by a record indicating if the line was a definition or a statement and if statement then if it contained a label and what was the value of the location counter was before processing that line.
- The use of such kind of intermediate files can eliminate unnecessary computations in the two-pass assembler but then it adds to the input-output burden of the system.